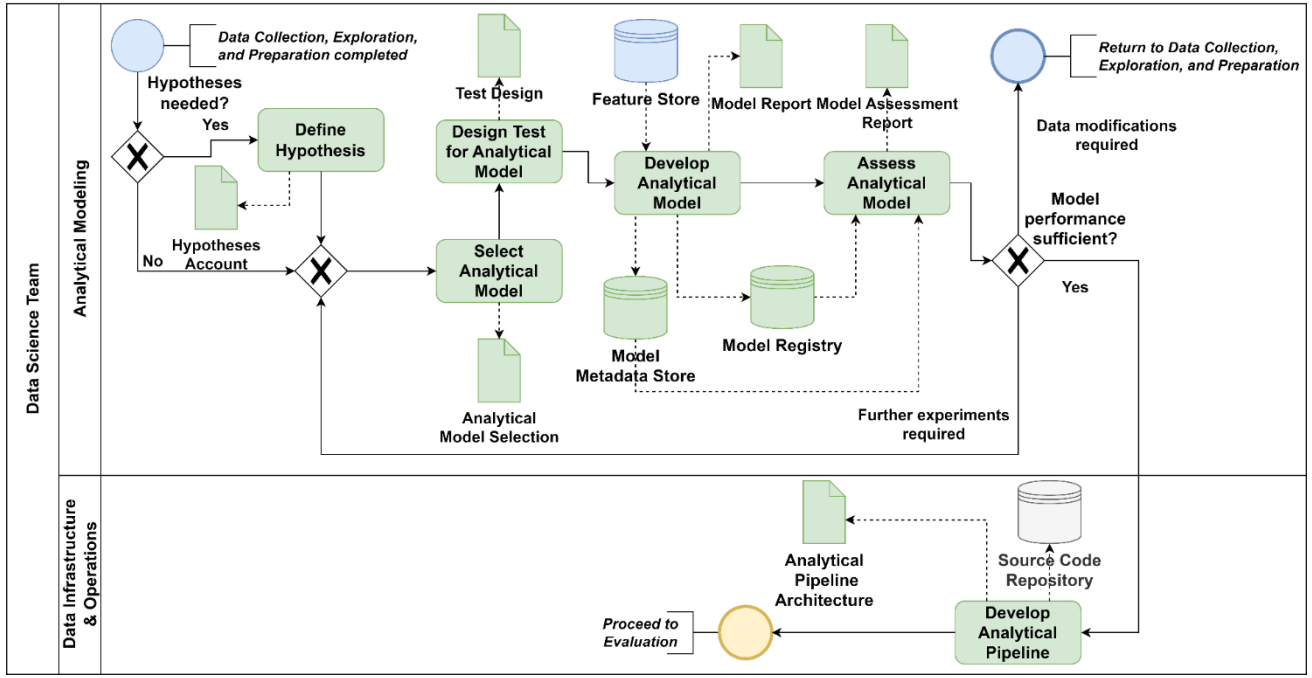
Modeling

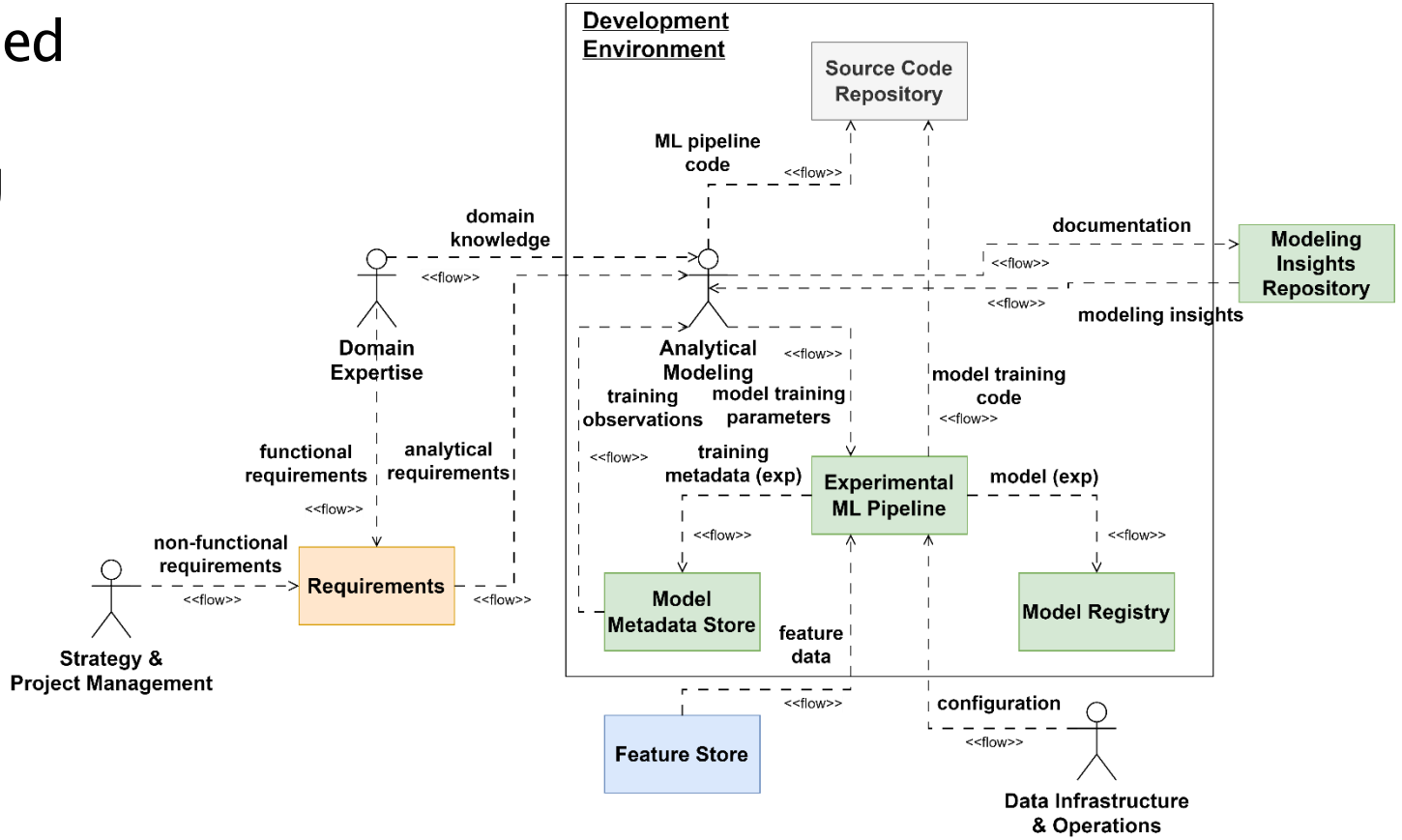
Modeling – Process Perspective

- Select, train, and technically evaluate analytical models to fulfill the DS targets
- Main tasks
  - (Define Hypothesis)
  - Select Analytical Model
  - Design Test for Analytical Model
  - Develop Analytical Model
  - Assess Analytical Model
  - Develop Analytical Pipeline
- Artifacts
  - Hypotheses Account
  - Analytical Model Selection
  - Test Design
  - Model Report
  - Model Assessment Report
  - Analytical Pipeline Architecture



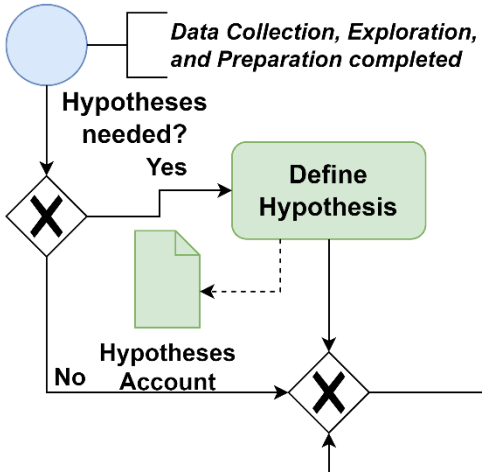
Modeling – Information Flow Perspective

- Shaped by analytical requirements
- Experimental ML pipeline supplied by the feature store
- Automated experiment tracking and model storing
- MLOps components
  - Source code repository
  - Experimental ML pipeline
  - Metadata store
  - Model registry
- Modeling Insights Repository
  - Collection of documentation artifacts



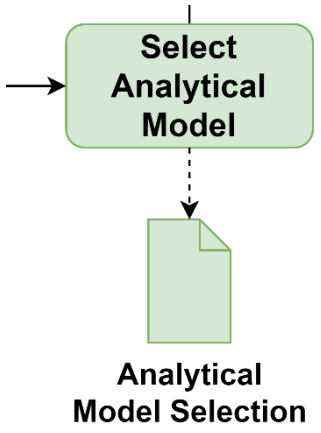
Define Hypothesis

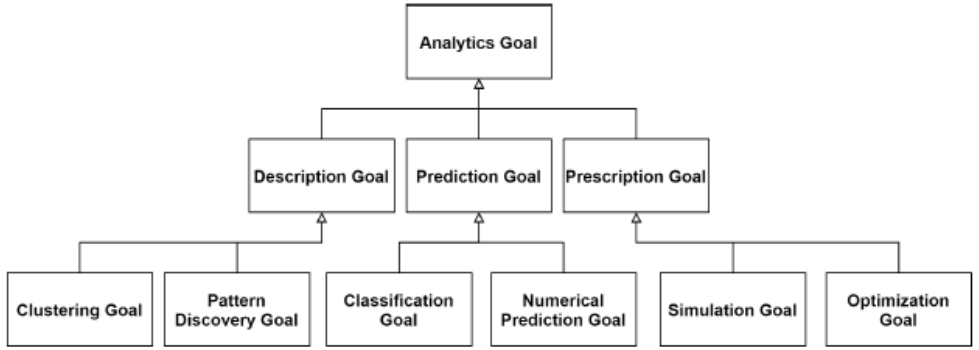
- Goal: formulate hypotheses based on insights from data exploration and preparation to guide experimentation (optional)
  - Particularly useful for statistical tests
- Hypotheses Account (Moyle et al. 2001)
  - Documents the defined hypotheses
  - Includes the hypotheses, (counter)arguments, tests to validate/reject, and their refinement/generalization



Select Analytical Model

- Goal: identify suitable analytical algorithms for the business and Data Science goal at hand in a systematic and structured way
- Appropriate algorithms result from defined requirements
  - Examples: problem type, data, available computing resources, explainability, etc.
- Adequate documentation ensures reproducibility
- Analytical Model Selection
  - Lists and compares potential candidates according to defined criteria
  - Conclusion and selection of suitable algorithm(s)

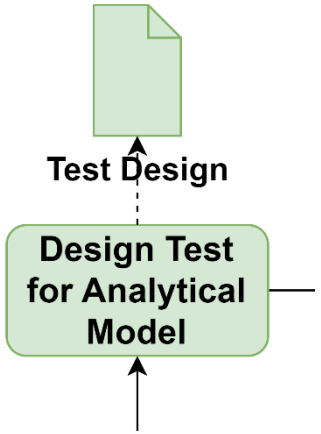




Taxonomy of analytical problem types (Nalchigar 2020)

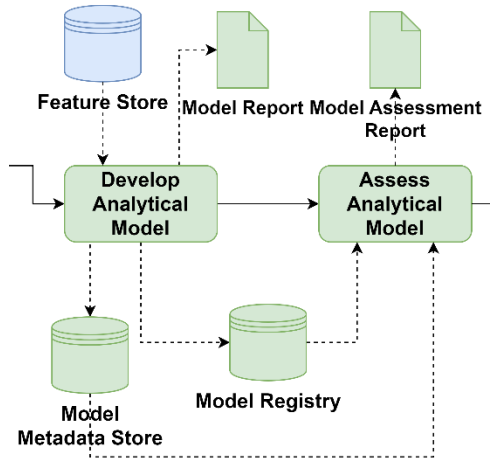
Design Test for Analytical Model

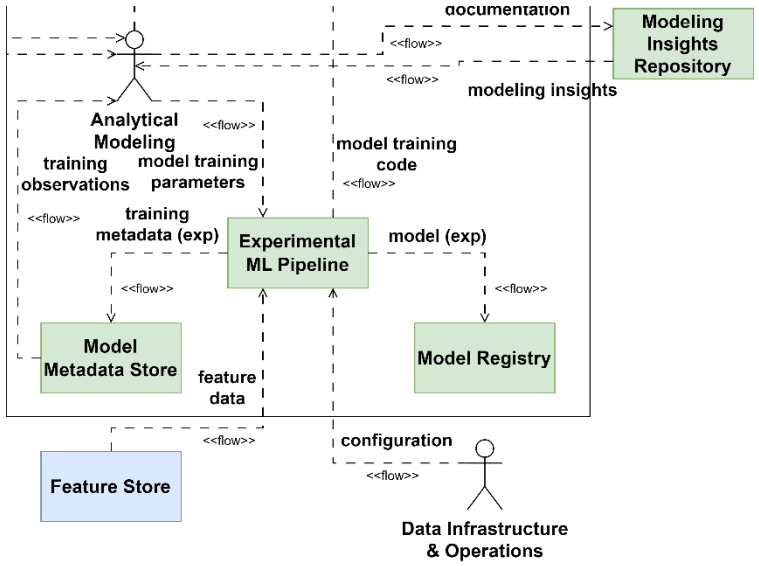
- Goal: define how selected algorithms are created and evaluated
- Train–Test split and subsequent feature engineering
  - Mapping/encoding: mapping values from one or more columns to new column values using user–defined operations or code
  - numerical transformations: log transformation, scaling (e.g., min–max, z–score standardization
  - ML methods: clustering, dimensionality reduction, regression, classification, imputing, embedding
  - …
- Considers particularities of the algorithm and the Data Science targets
- Serves as valuable input for shaping metadata tracking during experimentation
- Test design
  - input data (e.g., origin, distribution)
  - Feature engineering activities
  - Evaluation metrics, derived from the Data Science targets and success criteria
  - inputs of the domain and data experts for assessing the models



Develop Analytical Model

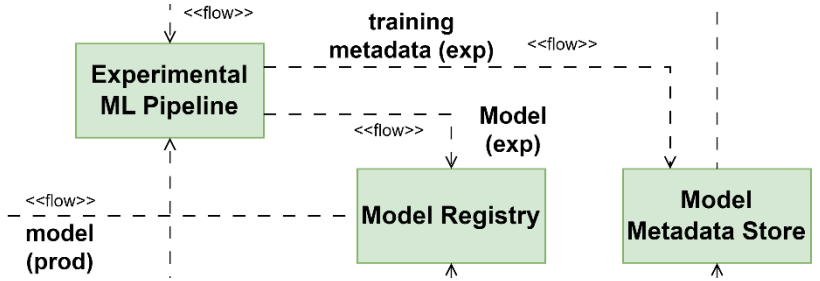
- Goal: train analytical algorithms
  - Often an experimental and iterative process
- Levers
  - Data transformations (e.g., features used)
  - Training algorithms and hyperparameters
    - Hyperparameter: tunable values controlling the learning process
      - Set before training a model (e.g., neural network architecture, decision tree branches, learning rate)
      - Tuning methods: Grid Search, Random Search, Bayesian Optimization
  - Evaluation metrics
- MLOps components to ensure reproducibility: Feature Store, Model Registry, and Model Metadata Store
- Model Report
  - Created for every model
  - Model type, target variable, used data, and features
  - Results and interpretations
  - Model performance (metrics) according to test design





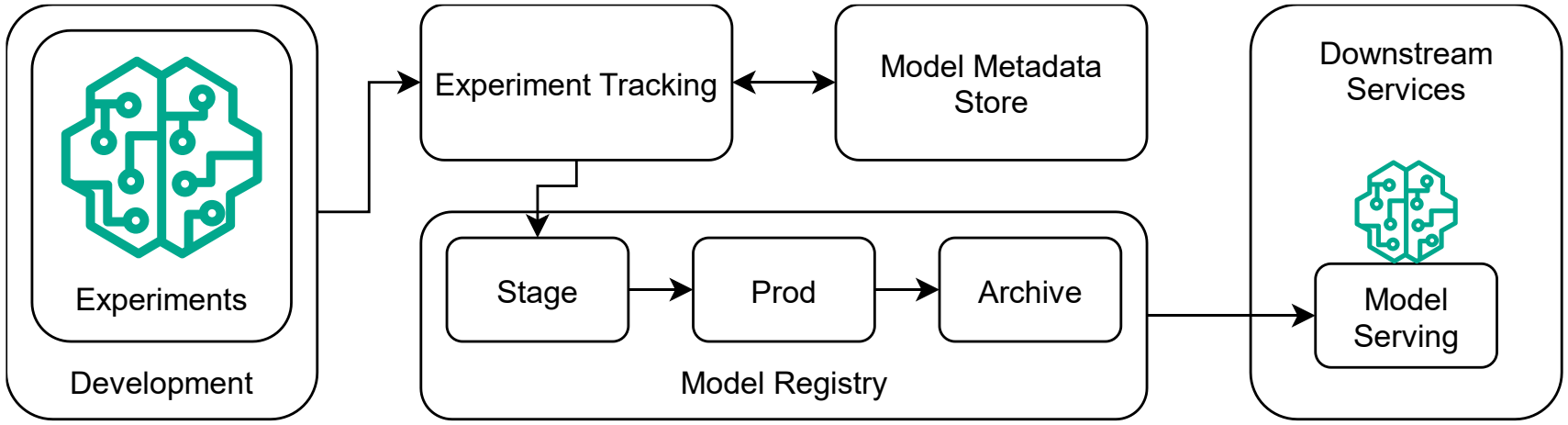
Model Metadata Store

- Centralized place to manage MLOps metadata
- Motivation
  - Difficult and cumbersome to manually track experiments and results
  - Resource waste (e.g., time)
  - Lack of reproducibility and sharing of results
- Automated logging includes:
  - Code, environment, data, (hyper)parameters, metrics
  - Model pipeline metadata
- Benefits
  - Reproducibility of experiments
  - Tracking system performance → informed decisions
  - Identification of potential issues for improvement
  - Creates transparency → increases trustworthiness of systems
- Tools: TensorBoard, MLFlow, Weights & Biases, Neptune



Model Registry

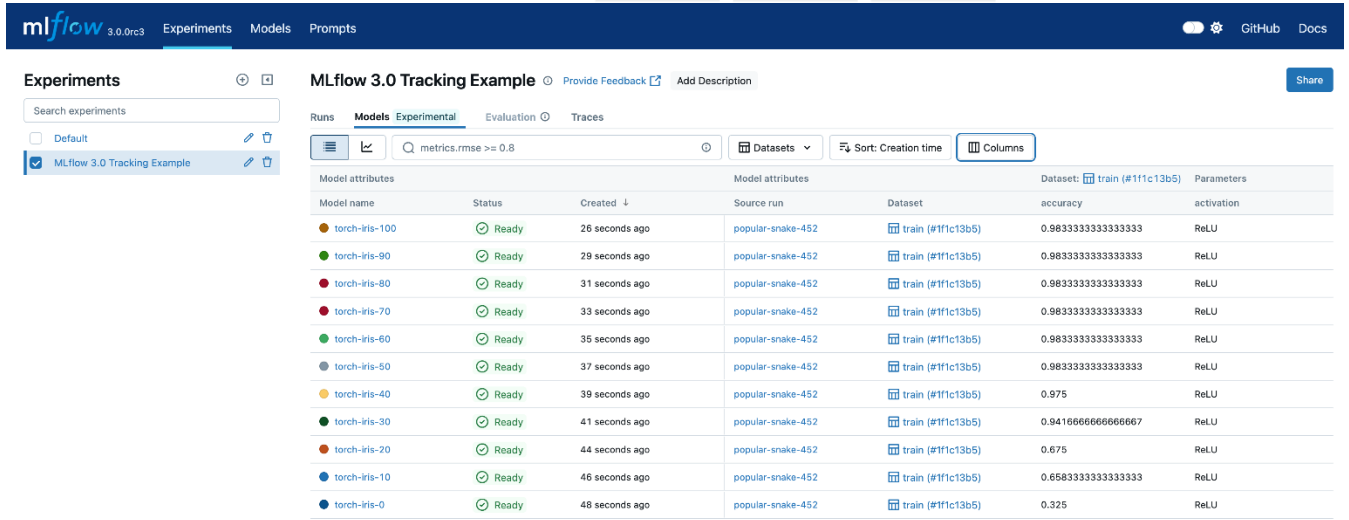
- Central storage of trained ML models and their metadata
  - Enables model version control and deployment management
  - Debugging and collaboration
- Model storage formats (examples)
  - PMML (Predictive Model Markup Language)
  - Pickle (pkl, joblib)
  - Docker images
- Tools: MLFlow, Amazon SageMaker Model Registry, Google VertexAI Model Registry

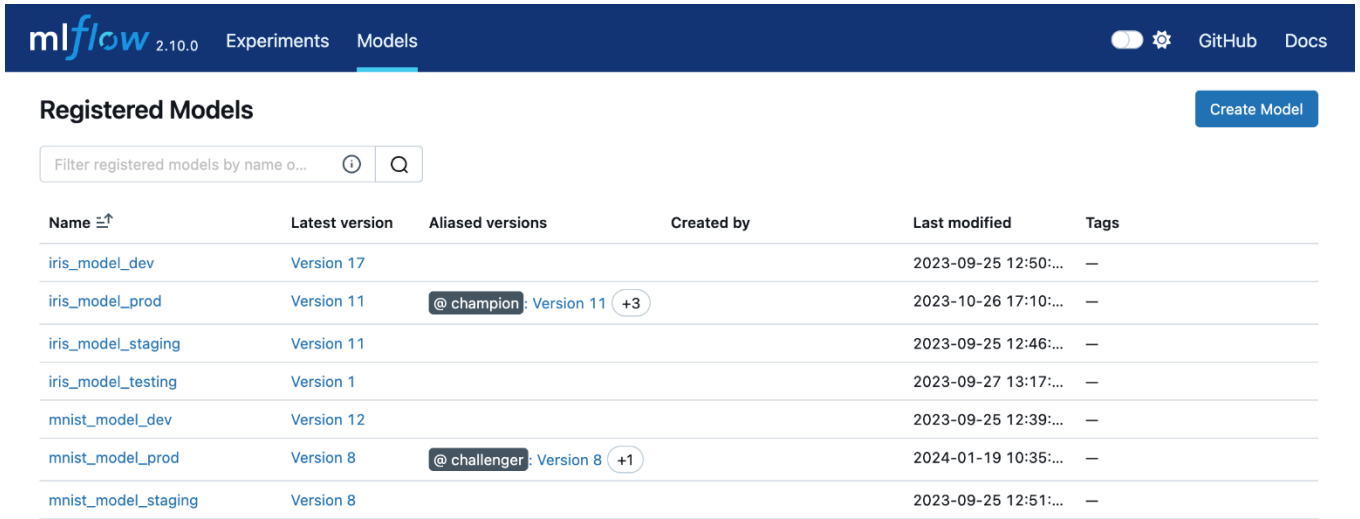


Role of the Model Registry

MLflow

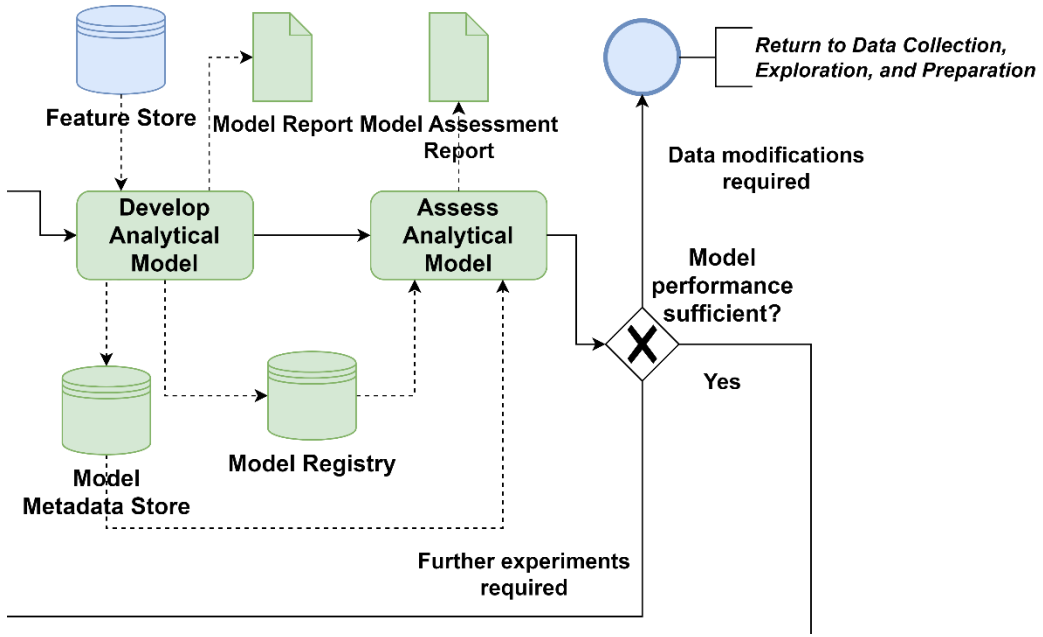
- Mlflow Tracking
  - Record metrics and parameters from training runs
  - Query data from experiments
  - store models, artifacts, and code
- Model Registry
  - Store and version ML models
  - Load and deploy ML models
- ...





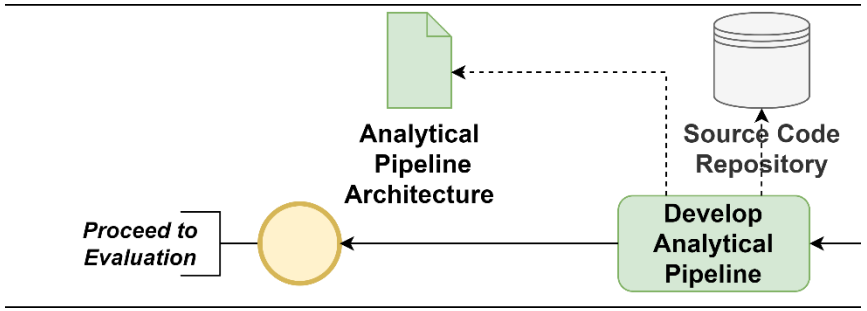
Assess Analytical Model

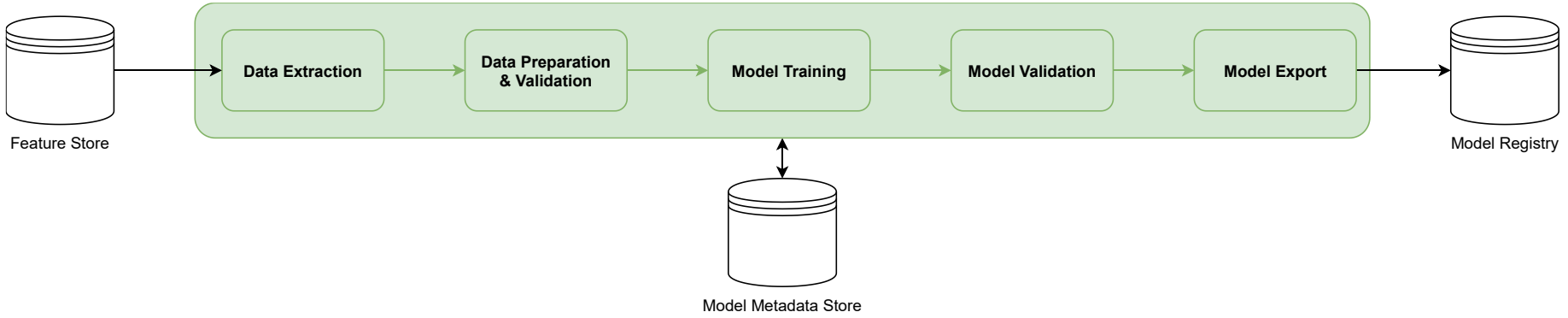
- Goal: evaluate the trained analytical models in relation to the Data Science objectives
- Perspectives (examples)
  - Hyperparameters
  - Model overfitting
  - Baseline comparison
  - Feature importance
  - Computational intensity
- Model Assessment Report
  - Interpretation of model performance metrics
  - Results of model tests
  - Input of domain and data experts
  - Explanation of the results
  - Conclusions for continuation of the project



Develop Analytical Pipeline

- Goal: develop an ML production pipeline to enable automation of model (re)training and serving in production
- MLOps relies on deploying an entire ML pipeline instead of individual models
  - Motivation: differences between environments + automation
- Benefits
  - Modularity & reusability
  - Automation of model deployment → consistency and reliability
  - Quality assurance
- Analytical Pipeline Architecture
  - Describes the architecture of the pipeline visually or textually

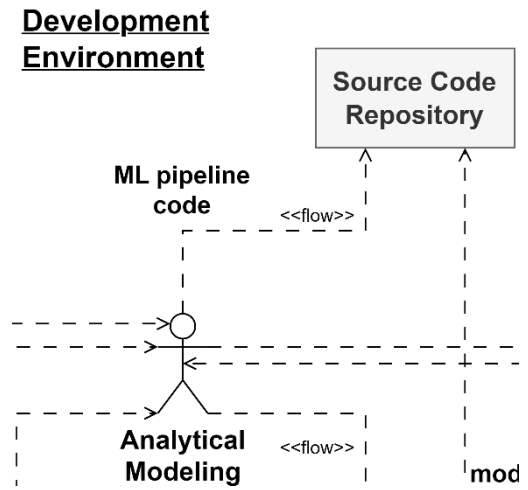




Exemplary model pipeline in MLOps (Kreuzberger et al. 2023)

Source Code Repository and CI/CD (Model Pipeline)

- Purpose: multiple developers can contribute model pipeline code updates to a shared repository, conduct automated testing, and enable a controlled and continuous deployment process
  - CI: frequent code merging and early issue detection
  - Continuous Delivery: deployment with manual approval
  - Continuous Deployment: automating deployment without manual intervention
    - Testing: beyond traditional functional and unit testing
    - CD challenges: complexities in model serving, monitoring, and updates
- Versioning extends beyond code and incorporates data source objects, parameters, and multiple artifacts
  - Model versioning: Model Registry
  - Data versioning: Feature Store



Model and ML Pipeline Testing

- Testing is complex in ML systems, include data, infrastructure, model, prediction, system, etc.
- Model (pipeline) testing
  - Unit tests: independent units within a pipeline step behave as expected
  - Integration tests: interaction of pipeline steps works appropriately
  - Smoke tests: basic checks to confirm plausibility of pipeline/model (intermediate) outputs
  - Load tests: testing model pipeline under heavy conditions
  - Metamorphic tests: feature adjustment influences model prediction accordingly (model robustness)
  - Reproducibility testing: similar results when using the same model
  - …

References

- Haertel, C., Pohl, M., Staegemann, D., & Turowski, K. (2022). Project Artifacts for the Data Science Lifecycle: A Comprehensive Overview. 2022 IEEE International Conference on Big Data (Big Data).
- Haviv, Y., Gift, N. (2023). Implementing MLOps in the Enterprise. O'Reilly Media, Inc.
- https://app.datacamp.com/learn/skill-tracks/mlops-fundamentals
- https://mlflow.org/docs/latest/ml/
- Kreuzberger, D., Kühl, N., & Hirschl, S. (2023). Machine Learning Operations (MLOps): Overview, Definition, and Architecture. IEEE Access, 11, 31866-31879.
- Moyle S. and Jorge A. (2001): "RAMSYS – A methodology for supporting rapid remote collaborative data mining projects".
- Nalchigar, S. (2020): From Business Goals to Analytics and Machine Learning Solutions: A Conceptual Modeling Framework.